

Crea un protocolo de revisión de código para comprobar la seguridad contra inyecciones SQL en un backend. Explica qué aspectos hay que revisar y cómo se pueden evitar estas vulnerabilidades.

Elaboración de un protocolo de revisión de código contra inyección SQL.

Elaboración de un protocolo de revisión de código contra inyección SQL.

Protocolo de revisión de código: inyección SQL (SQLi)

1. Objetivo y alcance

Verificar que ningún dato controlado por el usuario (o por un sistema externo) llegue a una consulta SQL como parte de su estructura, sino solo como dato.

Alcance: controladores/endpoints, capa de servicio, repositorios/DAOs, ORM, procedimientos almacenados, migraciones, scripts de reporting y jobs que accedan a la base de datos.

2. Fase 1: Mapear fuentes y destinos

Fuentes de datos no confiables (no solo los formularios):

- Query string, path params, body (JSON/XML/form), cabeceras, cookies
- Ficheros subidos (CSV, Excel) y su contenido
- Mensajes de colas, webhooks, APIs de terceros
- Datos ya almacenados en la BD (inyección de segundo orden)

Destinos (sinks): todo lugar donde se ejecute SQL: `execute`, `query`, `raw`, `createNativeQuery`, `exec`, `prepare`, `EXECUTE IMMEDIATE`, etc.

Tarea: seguir el flujo de cada fuente hasta cada destino y anotar qué transformaciones sufre por el camino.

3. Fase 2: Qué revisar

3.1 Construcción dinámica de consultas

- ¿Hay concatenación (`+`, `.concat`), interpolación (`f"...", `${}``, `String.format, %s`) o formateo de cadenas dentro de una sentencia SQL?
- ¿Se construyen cláusulas `WHERE` por partes con datos del usuario?

python

❌ Vulnerable

```
cursor.execute(f"SELECT * FROM users WHERE email = '{email}'")
```

✅ Seguro (parametrizada)

```
cursor.execute("SELECT * FROM users WHERE email = %s", (email,))
```

3.2 Uso incorrecto de ORMs y query builders

Los ORMs protegen solo si se usan como deben. Revisar:

- Métodos "raw" o nativos: `raw()`, `extra()`, `createNativeQuery`, `sequelize.query`, `knex.raw`, `@Query(nativeQuery=true)`, `FromSqlRaw`.
- Concatenación dentro de fragmentos HQL/JPQL.
- Filtros contruidos a partir de diccionarios/JSON enviados por el cliente (p. ej., `filter(**request.GET)`), que permiten elegir campos y operadores.

java

```
// ❌ Vulnerable
em.createQuery("FROM User u WHERE u.name = '" + name + "'");

// ✅ Seguro
em.createQuery("FROM User u WHERE u.name = :name").setParameter("name",
name);
```

3.3 Identificadores dinámicos (tablas, columnas, ORDER BY)

Los parámetros no sirven para identificadores. Es el caso más frecuente de falsa sensación de seguridad.

- ¿El nombre de columna u orden de clasificación viene del cliente?
- ¿Se valida contra una lista blanca fija?

python

```
ALLOWED_SORT = {"name": "name", "created": "created_at"}
col = ALLOWED_SORT.get(sort_param, "created_at")
dir_ = "DESC" if direction == "desc" else "ASC"
query = f"SELECT * FROM products ORDER BY {col} {dir_}"
```

3.4 Casos especiales

- **LIKE**: el parámetro debe ir enlazado (`LIKE ?` con `"%"+valor+"%"` como valor), no concatenado en la sentencia.
- **IN (...)**: generar los placeholders según el número de elementos (`?, ?, ?`) y enlazar cada valor; no unir los valores en una cadena.
- **LIMIT/OFFSET**: castear a entero y acotar rangos.
- Inserciones masivas y **UPDATE** dinámicos: revisar que los nombres de columnas no provengan de claves del JSON del usuario.

3.5 Procedimientos almacenados y SQL dinámico en la BD

- Buscar `EXEC(@sql)`, `EXECUTE IMMEDIATE`, `sp_executesql` sin parámetros, `PREPARE` con concatenación.
- Un procedimiento almacenado no es seguro por sí mismo si construye SQL con cadenas.

3.6 Inyección de segundo orden

- ¿Se reutilizan datos leídos de la BD (nombres de usuario, comentarios) para construir nuevas consultas sin parametrizar? Que un dato esté "en nuestra base de datos" no lo hace confiable.

3.7 Validación y sanitización

- La validación (tipo, longitud, formato, rango) es una capa adicional, no sustituye a la parametrización.
- Desconfiar de funciones de escape manuales (`replace("'", "''")`, `addslashes`): son frágiles y dependen del charset.
- Comprobar que los IDs numéricos se convierten realmente a entero antes de usarse.

3.8 Configuración y entorno

- Privilegios de la cuenta de BD: ¿la app usa un usuario con el mínimo necesario (sin `DROP`, `FILE`, `xp_cmdshell`, ni acceso a otros esquemas)?
- Separación de cuentas para lectura/escritura/migraciones, si procede.
- Manejo de errores: que no se devuelvan al cliente trazas, mensajes del motor ni la consulta ejecutada.
- Logs: no registrar credenciales ni datos sensibles; sí registrar errores de BD anómalos.
- Consultas apiladas (multi-statement) deshabilitadas en el driver si no son necesarias.

3.9 Otras bases de datos

Si hay NoSQL (MongoDB, etc.), revisar inyección de operadores (`{"$ne": ""}`) y construcción de filtros a partir de JSON del cliente.

4. Fase 3: Búsqueda sistemática (ayudas)

Patrones para localizar sinks sospechosos (adaptar al lenguaje):

bash

```
# Concatenación o interpolación cerca de palabras clave SQL
grep -rnE
"(SELECT|INSERT|UPDATE|DELETE|WHERE).* (\\+|\\$\\{|%s|format\\(|f\\)" " src/

# Llamadas "raw"/nativas
grep -rnE
"(raw\\(|createNativeQuery|nativeQuery|FromSqlRaw|sequelize\\.query|execute\\()" " src/
```

Herramientas automáticas (SAST): Semgrep (reglas `sql-injection`), CodeQL, SonarQube, Bandit (Python), SpotBugs/Find Security Bugs (Java), gosec (Go), Brakeman (Rails). Revisar siempre los resultados manualmente: hay falsos positivos y negativos.

5. Cómo evitar estas vulnerabilidades

Prioridad	Medida	Detalle
1	Consultas parametrizadas / prepared statements	Separan código SQL de datos. Es la defensa principal.
2	ORM usado correctamente	Evitar métodos raw; si son inevitables, parametrizar.
3	Listas blancas para identificadores	Para <code>ORDER BY</code> , nombres de tabla/columna, direcciones de orden.
4	Procedimientos almacenados seguros	Con parámetros; sin SQL dinámico concatenado.
5	Validación de entrada	Tipos, longitudes, formatos, rangos. Defensa en profundidad.
6	Mínimo privilegio en la BD	Limita el impacto si algo falla.
7	Errores genéricos al cliente	Evita fugas de información sobre el esquema.
8	WAF	Capa complementaria, nunca sustituto de arreglar el código.

6. Fase 4: Verificación

- Tests unitarios/de integración con payloads típicos (`' OR '1'='1, ';' --`, comillas, caracteres Unicode) sobre cada endpoint que toque la BD.
- Tests de regresión para cada hallazgo corregido.
- DAST/pentest (OWASP ZAP, Burp Suite, sqlmap) solo en entornos de prueba y con autorización.
- Integración en CI/CD: SAST obligatorio en cada pull request, bloqueando si hay hallazgos críticos.

7. Clasificación de hallazgos

Severidad	Criterio	Ejemplo
Crítica	Entrada sin autenticar llega a SQL concatenado	Endpoint público con <code>WHERE id = + param</code>

Alta	Igual, pero con autenticación; o <code>ORDER BY</code> sin lista blanca	Panel de usuario con ordenación libre
Media	Raw query con parámetros no controlados por el usuario hoy, pero frágil	Constante concatenada que podría cambiar
Baja/Inf o	Buenas prácticas incumplidas	Cuenta de BD con más privilegios de los necesarios

8. Checklist rápida del revisor

Todas las consultas usan parámetros enlazados
 No hay concatenación/interpolación de datos externos en SQL
 Los usos de `raw/native` están justificados y parametrizados
`ORDER BY`, tablas y columnas dinámicas usan lista blanca
`LIKE`, `IN`, `LIMIT` están tratados correctamente
 Los procedimientos almacenados no construyen SQL por concatenación
 Se considera la inyección de segundo orden
 La cuenta de BD tiene privilegios mínimos
 Los errores de BD no se exponen al cliente
 Hay tests y SAST en el pipeline
 Los hallazgos están clasificados, asignados y con tests de regresión

Si quieres, puedo adaptarlo a un stack concreto (Node/Express, Django, Spring, .NET...) con ejemplos y reglas de Semgrep específicas, o convertirlo en un documento descargable.